FINAL REVIEW · PROJECT PRESENTATION

PRISM

Predictive, explainable system-health monitoring on a personal computer

Sahil Dhanuka
Reg. No. 24BDS0456

Shambhav Ghosh
Reg. No. 24BCB0013

Naresh Kumar Bishnoi
Reg. No. 24BDS0336

October 2026

Monitoring tools react; PRISM predicts and explains

Activity Monitor / Task Manager today

- Shows usage only after the slowdown is felt
- Raw numbers, no cause
- No advice on what to do

PRISM answers four questions every 5 s

1. Where will CPU and memory be in 45 s and 210 s?
2. How likely is a slowdown (resource contention)?
3. What is causing it?
4. What should the user do?

How PRISM works, on the laptop itself

Collect
CPU, memory, disk, network and processes

→

Forecast
CPU and memory 45 s and 210 s ahead

→

Score risk
probability of a slowdown

→

Explain
the most likely cause

→

Advise
three concrete actions

How PRISM compares with existing work

	Activity Monitor / Task Manager built into macOS / Windows	PRESS Gong et al., CNSM 2010	ARIMA workload prediction Roy et al. 2011; Calheiros et al. 2015	PRISM (ours) this project
Where it runs	Personal computer	Cloud servers	Cloud servers	**Personal computer**
Predicts future load	✗ current only	✓	✓	**✓ 45 s and 210 s ahead**
Method	—	FFT + Markov chain	ARIMA time-series model	**LightGBM (beat RF, Ridge)**
Explains the cause	✗	✗	✗	**✓ SHAP**
Advice to the user	✗	auto-scales servers	auto-scales servers	**✓ 3 concrete actions**
Flags unusual behaviour	✗	✗	✗	**✓ Isolation Forest**
Trains on the user's machine	— (no model)	✗	✗	**✓ with a 5 % deployment gate**

Different setting
Cloud work sees smooth, aggregated demand. A laptop is mostly idle with short, user-driven bursts.

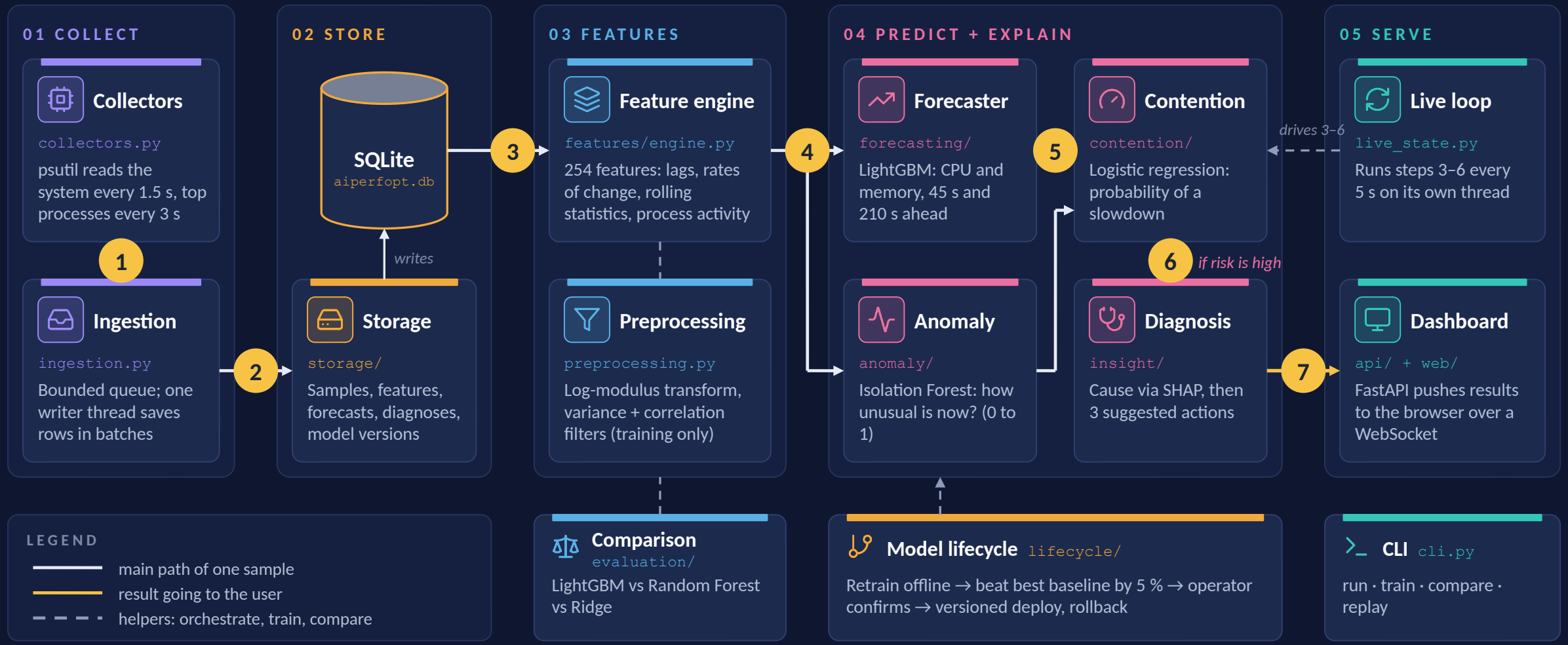
Beyond forecasting
PRISM adds the cause and the action, not just a number.

Private and local
Data, training and models never leave the machine.

ARCHITECTURE

From one OS sample to a forecast on screen

Follow the numbers 1 → 7



Standard time-series features, built without look-ahead

Table 1 **254 features per sample, by technique**

Feature group	Technique	Count	Example
Current value	raw observation	1	CPU % at time t
Lag features	lagged observations	16	value 30 s and 120 s ago
Rate of change	first difference ÷ Δt	8	slope per second
Rolling statistics	sliding-window mean, std, min, max	64	over 30 s and 120 s
Process activity	per-process aggregation	165	launches, top-1 / top-5 CPU share
Total		**254**	

TARGET: PREDICT THE CHANGE

$$\Delta = y(t + h) - y(t)$$

Differencing, as in the "I" of ARIMA. Forecast = current value + predicted change, so "no change" is the model's natural zero.

No look-ahead

Every feature uses only data at or before t. One function builds features for training and for live use; a test checks both give the same vector.

PREPROCESSING

Eight steps, each a standard method, fitted without leakage

Stage	Step	Established method	What it does in PRISM	Effect
Per sample	**Clean timestamps**	**Sort + de-duplicate**	later sample wins on a repeated timestamp	—
Per sample	**Counters → rates**	**First differencing**	cumulative bytes → bytes / s; counter reset or gap → missing	—
Rows	**Validity filter**	**Domain range check**	drop non-finite values or values outside 0–100 %	0 rows
Rows	**Warm-up removal**	**Gap segmentation + burn-in removal**	new segment after a gap > 30 s; drop its first 120 s	988 rows
Columns [1]	**Skewed byte rates**	**Log-modulus transform (John & Draper, 1980)**	$\text{sign}(x) \cdot \log(1 + \lvert x \rvert)$	44 columns
Columns [1]	**Constant features**	**Zero-variance filter**	drop columns with a single value	1 dropped
Columns [1]	**Redundant features**	**Pearson correlation filter**	if $\lvert r \rvert > 0.99$, drop the later column	29 dropped
Per model	**Missing values, scale**	**Trees: native missing-value splits · Ridge: median imputation + z-score (StandardScaler)**	LightGBM / RF route gaps themselves; Ridge needs complete, scaled input	—

[1] *Fitted on training rows only, then applied unchanged to test rows (and refitted inside every cross-validation fold), so no test data influences which columns are kept. Overall: 254 → 224 features. Effects shown for the 45 s target.*

ALGORITHMS COMPARED

Three ML families, one fair test, a rule fixed in advance

LightGBM

Gradient-boosted trees

Each small tree fixes the errors of the previous ones

80 rounds · 15 leaves · lr 0.1

Random Forest

Bagged trees

Averages many deep, independent trees

100 trees · depth 12

Ridge

Regularised linear

Weighted sum of features, L2-penalised

α = 1 · imputed + scaled

Persistence

Reference, not a candidate

Predicts "no change": ŷ(t+h) = y(t)

the bar every model must beat

How they are tested

1. Chronological hold-out: oldest 80 % train, newest 20 % test
2. Walk-forward cross-validation (TimeSeriesSplit, 4 folds)
3. Same preprocessed features and same Δ target for all
4. 47.7 h of real usage from one Mac; nothing simulated

SELECTION RULE · written before the first run

Skill = 1 − MAE(model) / MAE(persistence)

Highest average skill over 4 targets (CPU, memory × 45 s, 210 s) wins. Skill above 0 means it beats "no change".

IMPLEMENTATION

Where each model lives in the code

All paths under src/aiperfopt/

Forecasters compared

LightGBM lgb.train, 80 rounds · 15 leaves	forecasting/model.py
Random Forest RandomForestRegressor, 100 trees	evaluation/model_comparison.py
Ridge median impute → scale → Ridge	evaluation/model_comparison.py

Baselines to beat

Persistence "no change": ŷ = current value	forecasting/baselines.py
Moving average trailing rolling mean	forecasting/baselines.py
Holt smoothing statsmodels Holt, linear trend	forecasting/baselines.py

Live risk and explanation

Isolation Forest how unusual is now (0 to 1)	anomaly/detector.py
Logistic regression probability of a slowdown	contention/model.py
SHAP TreeExplainer on LightGBM → cause	insight/explainer.py

Training and deployment

Train forecasters 4 LightGBM models	forecasting/training_pipeline.py
Train classifier logistic regression	contention/training_pipeline.py
Gate beat best baseline by 5 %	lifecycle/gate.py
Deploy / roll back versioned models	lifecycle/deployment.py

PRISM's forecasts are about 98 % accurate

OVERALL FORECAST ACCURACY

97.9 %

LightGBM, average of the 4 forecasts

HOW IT IS CALCULATED

Accuracy = 100 % − MAE

$$MAE = (1/n) \cdot \Sigma \mid forecast - actual \mid$$

The average miss in percentage points, over n ≈ 22,000 test samples the model never saw.

CPU 45 s: 100 − 2.95 = 97.0 %
Overall: 100 − 2.06 (mean MAE of 4) = 97.9 %

CPU 45 s ahead

97.0 %

Average error **±3.0 points**
Direction right **54 %**

CPU 210 s ahead

96.3 %

Average error **±3.7 points**
Direction right **54 %**

Memory 45 s ahead

99.4 %

Average error **±0.6 points**
Direction right **86 %**

Memory 210 s ahead

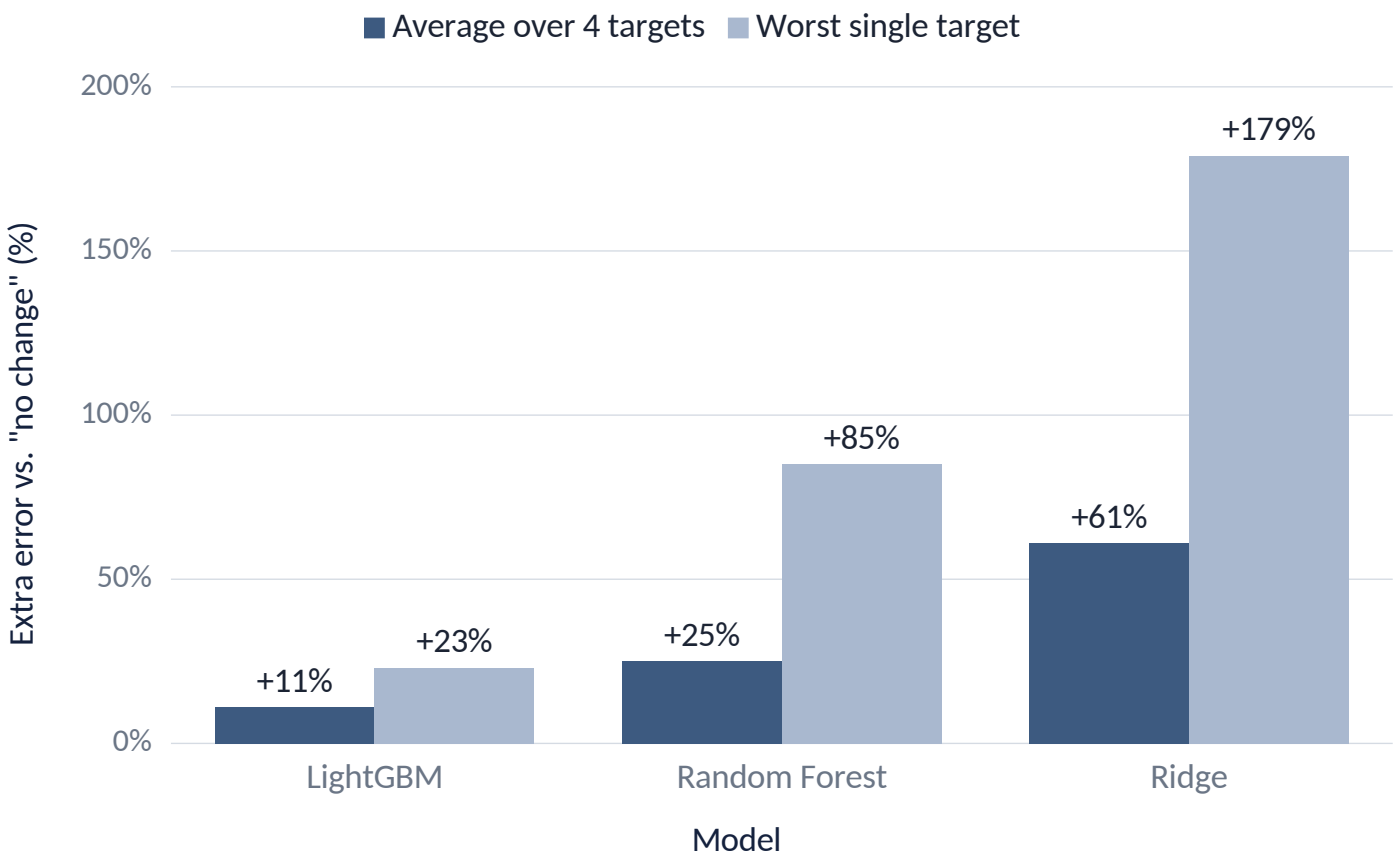
99.1 %

Average error **±0.9 points**
Direction right **82 %**

Direction right = share of samples where the forecast got up, down or steady right (steady = within 2 % of the current value). Memory is steadier than CPU, so it is easier to forecast.

LightGBM: smallest error, on average and at worst

Fig. 1 **Extra forecast error compared with "no change" (lower is better)**



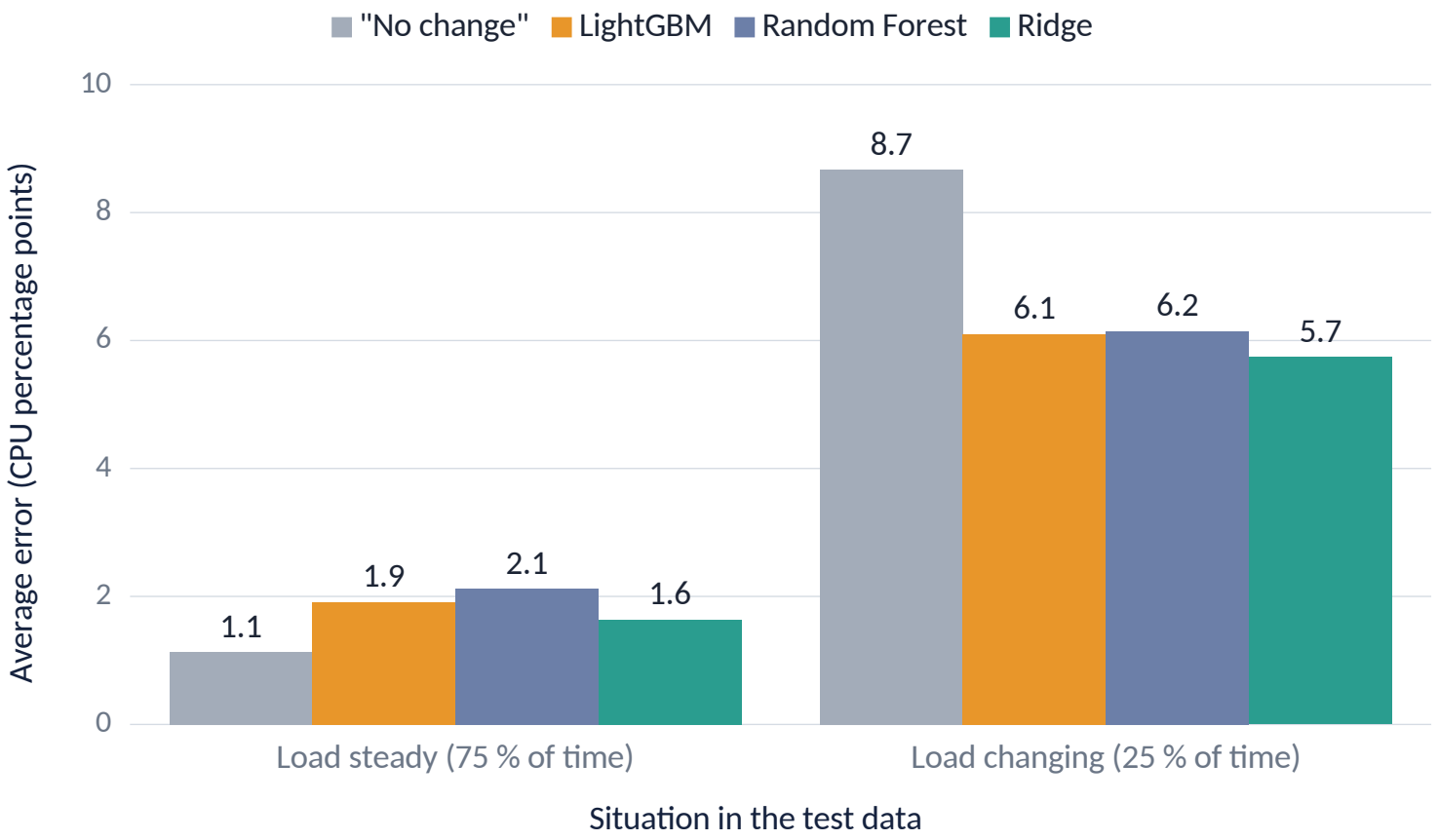
LightGBM
11 % more error than "no change" on average, never more than 23 %

Random Forest, Ridge
each wins one CPU target, but miss badly on memory (up to +85 % and +179 %)

Why all are above 0
the laptop was idle 97 % of the time, so "no change" is usually right; next slide shows where the models help

When load changes, the models cut error by ~30 %

Fig. 2 **Average CPU forecast error, 45 s ahead, split by how much the load changed**



−30 %

Load changing: LightGBM error 6.1 vs 8.7 points for "no change". This is when a warning is useful.

+69 %

Load steady: models predict small moves that don't happen (1.9 vs 1.1 points). 3 in 4 samples are steady, so the average looks worse.

Why PRISM uses LightGBM

1. **Most accurate overall**
   Best average skill under the rule fixed in advance.

2. **Never badly wrong**
   At most 23 % worse than "no change"; a monitor that is sometimes far off loses trust.

3. **Cheap on a laptop**
   Fast to predict and 15× faster to train than Random Forest, so retraining on the device is practical.

4. **Fits the system**
   Handles missing values after gaps, and SHAP explains each prediction exactly.

Table 2 **Running cost on the laptop (lower is better)**

	LightGBM	Rand. Forest	Ridge
Time per prediction	**1.7 ms**	15.8 ms	2.1 ms
Training time per target	**≈ 6 s**	≈ 88 s	**≈ 6 s**
Model file size	138 kB	4.5–10 MB	**17 kB**

Worth testing next: Ridge was best on short-term CPU. A per-target mix (Ridge for CPU, LightGBM for memory) may do better and needs confirming on new data.

DEPLOYMENT GATE

A model is served only if it beats the simple baselines

1 Retrain

on the user's own accumulated data, offline

→

2 Gate

must beat the best simple baseline (no change, moving average, Holt) by ≥ 5 %

→

3 Confirm

an operator approves in the CLI, API or Model view; never automatic

→

4 Deploy

versioned, stored with its evidence; one click rolls back

If the gate fails, the best baseline keeps serving.

Served today: moving average

On the current data LightGBM does not yet clear the 5 % gate, so the gate correctly keeps it out until it earns its place on new data.

CONCLUSION

1. **A complete on-device monitor**
   Predicts, explains and advises, with a gated, human-confirmed model lifecycle.

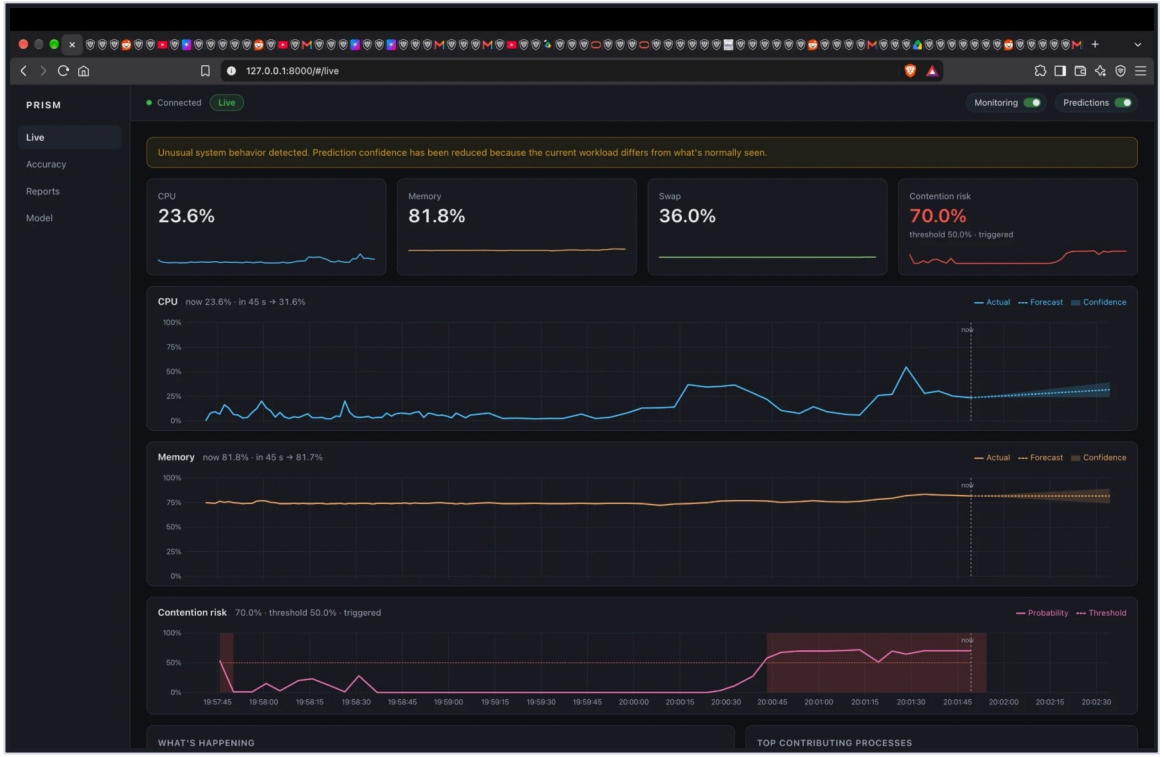
2. **A fair, leakage-free comparison**
   LightGBM chosen by a rule fixed in advance: smallest error on average and in the worst case.

3. **Value where it matters**
   When CPU load changes, the models cut forecast error by about 30 % versus "no change".

Thank you

Sahil Dhanuka · Shambhav Ghosh · Naresh Kumar Bishnoi

PRISM sees a slowdown coming and says why





1

Risk crosses the threshold

Contention risk jumps to 70 % (threshold 50 %) with memory at 82 %. The anomaly detector also flags the workload as unusual and lowers confidence.

2

Cause and advice

"Memory usage is climbing steadily". Top advice: close unused browser tabs. The browser's renderer processes lead the memory list.

The advice is followed and the risk goes away





3 **Tabs closed, memory drops**
Memory falls from about 82 % to 63 % at 20:02:20; contention risk drops to 0 % within seconds.

4 **All clear**
"Nothing needs your attention right now." The forecasts keep running every 5 s.